

- Due Mar 14 at 11:59pm
- Points 40
- Questions 61
- Time Limit None
- Allowed Attempts Unlimited

Instructions



Week 10

Final Exam



Instructions

1. This exam is a take-home exam.
 2. **All** work must be done independently. If you have questions, please reach out to me. Do not talk to anyone else about your exam or how you solved your problems.
 3. The exam will be available through the end of W13.
 4. This is an open textbook, open brain, and open Internet exam. You may take this exam as many times as you would like through the end of W13.
- NOTE:** You are not allowed to use other people (in class or out of class). You may use computer tools to check your answers but do not let that do the work for you!

Take the Quiz Again

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	5,732 minutes	24.28 out of 40 *

* Some questions not yet graded

Score for this attempt: 24.28 out of 40 *

* Some questions not yet graded

Submitted Mar 11 at 11:07am

This attempt took 5,732 minutes.



Question 1

Not yet graded / 4 pts

In the context of a real-time operating system (RTOS), both hardware timers and software timers are used to manage time-based operations. Discuss the **advantages and disadvantages** of each type of timer. In your response, explain how each timer works, compare their precision, scalability, and system resource usage, and describe scenarios where one might be preferred over the other.

Your Answer:

Hardware timers use separate modules designed specifically for the timer, like a real-time clock. Software timers use code and software routines, they count clock cycles or use delays. Hardware timers are generally more reliable, because they are independent of everything else going on in the processor. They are also more accurate. Software timers are easier to make, but less accurate because of how it relies on other parts of the system. Software timers are also more flexible than hardware timers, because the software routine can change pretty easily. The hardware can't change, and using it requires to change software anyways, like a prescaler or counter. Hardware timers don't use much of the system's resources, because of its independence. Software does, because of its dependence on the system. In a scenario where you need something super precise, with a small range of differences, use a hardware timer. In a scenario where you want to use lots of different values of timers easily, and the accuracy doesn't matter as much, use a software timer.



Question 2

Not yet graded / 4 pts

Explain the concept of *cycle-stealing* as it relates to Direct Memory Access (DMA) controllers in embedded systems. In your response, describe how cycle-stealing works, why it is used, and how it affects CPU processing time and overall system performance. Provide examples of application scenarios where cycle-stealing might be beneficial or problematic.

Your Answer:

Direct Memory Access (DMA) is where a peripheral device can access RAM directly, instead of having to have the CPU do it. This greatly reduces the stress of tending to peripheral devices on the CPU, due to the potentially large amount of I/O data to and from the device. To do this, the DMA controller needs to take control of the bus. There are a few ways that it can do this, including cycle-stealing. Cycle-stealing is where the DMA controller takes temporary control of the bus, transferring one word of data. It then lets go of the memory bus for the CPU to have again. The CPU can then access memory in between DMA data transfers. Cycle-stealing is good if you are wanting to let the CPU be more free to access data. It is bad if the peripheral device needs to have the fastest transfer rate possible, because cycle-stealing is a bit slower. It is slower because it needs to prepare the data for each transfer, which takes time.



Question 3

Not yet graded / 4 pts

In a real-time embedded system with three tasks of varying priorities—low (Control), medium (System Identification), and high (Alarm)—two shared resources (ADC and DAC) are protected by separate mutexes. The Control task locks ADC first, then DAC. The SI task locks DAC first, then ADC. Given this resource acquisition pattern, is the system susceptible to a deadlock condition?

Please explain why. If it is susceptible to deadlock, describe **two different methods** you could use to resolve it.

Your Answer:

Yes, this system has some susceptibility to deadlock. It is due to the order that the resources are locked by the mutex. Let's do a scenario. Let's say that the control task runs first. It locks the ADC, and then is probably doing some computations with the data from it. This means that nothing else can access the ADC now whilst it is doing this. Now, while this is happening, let's say that the system identification task runs. It locks the DAC, and is probably doing something with it. After this, the control task is done and looks for the DAC, but it is unavailable, so it waits. While it is waiting the SI task is done, looks for the ADC, it is not available, so it waits. Uh oh... we are in an infinite waiting state (deadlock).

Notice that I assumed that the unlocking happens after each task locks both resources. One method that the deadlock can be resolved is to unlock the resource before looking to lock the second. This will free up the first resource to the other task for usage, and will prevent the deadlock. However, this may not be the best. We might want to keep the first resource locked while we use the second, so that the data isn't messed up during the second computation.

The second way that I would fix this would be to set a specific locking order. Meaning, only lock the ADC and DAC resources in a specific order inside of the tasks. Let's say that we preset this order to be ADC, then DAC, and run the same scenario as before. The control task runs, and the ADC is locked. After, the SI task runs, but the ADC is being used. This task is probably put into the waiting state now by the scheduler. So it waits. The control task then locks the DAC, probably does some computation, and then lets go of both the ADC and the DAC. The SI task now has access to the ADC, and can do the same thing. Now there is no deadlock.

Correct answer



Question 4

1 / 1 pts

Which of the following interrupt conditions are commonly issued by a DMA controller to indicate the status of its buffer during data transfer operations?

- ☒ Transfer Complete, Half Transfer, and Transfer Error.
- ☐ Buffer Overflow, Data Ready, and Transfer Timeout.
- ☐ Read Acknowledge, Write Acknowledge, and Buffer Empty.
- ☐ Interrupt Pending, Data Valid, and Transfer Retry.

- ☐ Start Condition, Stop Condition, and Arbitration Lost.

DMA controllers typically generate interrupts to signal key events during data transfers. The most common conditions are **Transfer Complete**, indicating that the entire block has been successfully moved; **Half Transfer**, which signals that half of the buffer has been processed (useful for double-buffering); and **Transfer Error**, which alerts the processor to issues such as bus faults or misaligned accesses. These interrupts allow the processor to respond efficiently to the progress and problems of DMA operations without constant polling, improving system performance and responsiveness.

Correct answer



Question 5

1 / 1 pts

Which of the following best describes the role of an Interrupt Vector Table (IVT) in a microcontroller system?

- ☒ A data structure that maps interrupt sources to their corresponding handler addresses.
- ☐ A memory-mapped register that stores the priority levels of all interrupts.
- ☐ A hardware module that queues interrupts based on their urgency.
- ☐ A software routine that polls for interrupt flags and dispatches handlers.
- ☐ A peripheral interface that manages communication between the CPU and external devices.

The Interrupt Vector Table (IVT) is a data structure, typically located at a fixed memory address, that holds the addresses of all the interrupt service routines (ISRs) for a system. When an interrupt occurs, the processor uses the IVT to determine which ISR to execute by indexing into the table using the interrupt number. This mechanism allows for fast and deterministic interrupt handling, which is critical in real-time and embedded systems. Unlike polling or software dispatching, the IVT enables hardware-level redirection to the appropriate handler.

Correct answer



Question 6

1 / 1 pts

For our labs, the pre-scaler on the Seven-Segment timer was set to 799 and the period was set to 99. Given these pre-scaler value and reload values, what is the frequency of the timer interrupt used to refresh digits on the multiplexed seven-segment LED display? Recall that the system clock is 80 MHz.

- ☒ 1000 Hz
- ☐ 8000 Hz
- ☐ 1 Hz
- ☐ 125 Hz
- ☐ 250 Hz

To calculate the timer interrupt frequency, use the formula:

$$\text{Frequency} = \frac{\text{System Clock}}{(\text{Prescaler}+1) \times (\text{Reload}+1)}$$

Plugging in the values:

$$\frac{8,000,000}{(7999+1) \times (0+1)} = \frac{8,000,000}{8000} = 1000 \text{ Hz}$$

This means the timer generates an interrupt every 1 millisecond, which is ideal for refreshing each digit in a multiplexed display. This frequency ensures smooth visual output while allowing the processor to handle other tasks efficiently.

Wrong answer



Question 7

0 / 1 pts

Our pressure sensor with a sensitivity of 0.98 mV/kPa and an offset of 0.140 kPa at 0 V is connected to a 12-bit ADC with a 2.5 V reference. If the ADC reading is **0x1AF**, what is the actual pressure in kilopascals (kPa)?

268.64 (with margin: 0.01)

To convert the ADC reading to pressure, first convert the 12-bit ADC value (0x1AF = 431) to voltage using the 2.5 V reference:

$$(431/4095) \times 2.5 \text{ V} \approx 0.263 \text{ V}$$

Then convert this voltage to millivolts and divide by the sensor sensitivity (0.98 mV/kPa):

$$(263 \text{ mV}) / (0.98 \text{ mV/kPa}) \approx 268.50 \text{ kPa}$$

Finally, add the 0.140 kPa offset to get the actual pressure:

$$268.50 + 0.140 = 268.64 \text{ kPa}$$

This process ensures accurate pressure readings by accounting for both the ADC resolution and the sensor's offset.

Wrong answer



Question 8

0 / 1 pts

A pressure sensor has a sensitivity of 0.98 mV/kPa and an analog output range of 0–2.5 V. If 0 V corresponds to a minimum detectable pressure of 0.140 kPa, what is the **maximum pressure** (in kPa) that the sensor can measure?

2,551.16 (with margin: 0.01)

To find the maximum measurable pressure, use the sensor's sensitivity and output range. The total voltage span is 2.5 V, and with a sensitivity of 0.98 mV/kPa (or 0.00098 V/kPa), the pressure span is:

$$\frac{2.5 \text{ V}}{0.00098 \text{ V/kPa}} \approx 2551.02 \text{ kPa}$$

Adding the minimum pressure offset of 0.140 kPa gives:

$$2551.02 + 0.140 = 2551.16 \text{ kPa}$$

Rounded to two decimal places, the maximum pressure is approximately **2,551.16 kPa**. This accounts for both the sensor's voltage range and its offset from zero pressure.

Correct answer



Question 9

1 / 1 pts

In the context of a Real-Time Operating System (RTOS), semaphores are versatile synchronization primitives. What are the two primary ways semaphores are typically used?

- ☒ As signaling mechanisms and for mutual exclusion.
- ☐ As memory allocation tools and for task scheduling.
- ☐ As interrupt handlers and for priority inversion.
- ☐ As debugging aids and for stack management.
- ☐ As delay generators and for CPU load balancing.

Semaphores in RTOS environments are primarily used in two ways: **as signaling mechanisms** between tasks or between ISRs and tasks, and **for mutual exclusion** to protect shared resources. Binary semaphores are often used for signaling, while counting semaphores can manage access to multiple instances of a resource. When used for mutual exclusion, semaphores help prevent race conditions by ensuring only one task accesses a critical section at a time. These uses make semaphores essential for managing concurrency and resource sharing in embedded systems.

Correct answer



Question 10

1 / 1 pts

During a context switch in a microcontroller-based system, which three types of information are typically saved and restored to ensure correct task resumption?

- ☒ Program Counter, Stack Pointer, and CPU Registers
- ☐ Interrupt Vector Table, Program Counter, and Flash Memory Contents
- ☐ Stack Pointer, GPIO Configuration, and Clock Settings
- ☐ CPU Registers, UART Buffers, and Timer Values
- ☐ Program Counter, DMA Configuration, and ADC Calibration Data

A context switch involves saving the current state of a task so that it can be resumed later without loss of information. The three most critical pieces of information are the **Program Counter (PC)**, which

indicates where the task left off; the **Stack Pointer (SP)**, which tracks the task's call stack; and the **CPU Registers**, which hold temporary data and computation results. These elements are essential for restoring the task's execution context accurately. Other hardware configurations like peripheral states are typically not part of the core context unless explicitly managed by the operating system.

Correct answer



Question 11

1 / 1 pts

A pressure sensor has a sensitivity of 0.98 mV/kPa and is connected to a microcontroller using a 2.5 V reference ADC. What is the minimum number of bits required for the ADC to detect pressure changes as small as **0.25 kPa**?

14

14 (with margin: 0)

To detect a 0.25 kPa change, the ADC must resolve voltage changes of at least **$0.25 \text{ kPa} \times 0.98 \text{ mV/kPa} = 0.245 \text{ mV}$** .

A 12-bit ADC with a 2.5 V reference has a resolution of approximately 0.610 mV, which is too coarse to detect such small changes. To achieve at least 0.245 mV resolution, the ADC must have **at least 14 bits**, which gives a resolution of about 0.153 mV—sufficient to detect 0.25 kPa changes reliably.

Correct answer



Question 12

1 / 1 pts

Why is it generally unsafe to use a mutex within an Interrupt Service Routine (ISR) in a real-time operating system (RTOS)?

- ☒ Because mutexes can cause priority inversion and are not designed to be used in interrupt context.
- ☐ Because mutexes disable interrupts, which can lead to missed deadlines.
- ☐ Because mutexes consume more memory than semaphores.
- ☐ Because ISRs cannot access kernel objects in any RTOS.
- ☐ Because mutexes automatically restart tasks, which is unsafe in ISRs.

Mutexes are designed for use in task-level synchronization and often include features like **priority inheritance** to prevent priority inversion. However, ISRs do not have a task context and cannot block or be suspended, which is a core behavior of mutexes when the resource is unavailable. Attempting to use a mutex in an ISR can lead to undefined behavior or system instability. Instead, **semaphores (especially binary semaphores)** are the appropriate synchronization primitive for signaling from an ISR to a task.

Correct answer



Question 13

1 / 1 pts

Why is the use of interrupts essential for correctly displaying digits on multiplexed seven-segment LED displays (such as the onen used in our labs) in embedded systems?

- ☒ To ensure timely and consistent refreshing of each digit without blocking the main program execution.
- ☐ To allow the processor to directly control each segment continuously in a polling loop.
- ☐ To reduce the power consumption of the display by disabling unused segments.
- ☐ To enable the display to operate independently of the system clock.
- ☐ To allow the display to store digit values in non-volatile memory.

In a multiplexed seven-segment display, each digit is rapidly turned on and off in sequence to create the illusion of a steady display. This requires precise and periodic updates to each digit, which is best handled using a timer interrupt. The interrupt ensures that the display is refreshed at a consistent rate without requiring the main program to constantly manage the display logic. Without interrupts, the processor would need to use inefficient polling or delay loops, which would block other tasks and reduce system responsiveness.

Correct answer



Question 14

0.25 / 0.25 pts

What is the most popular language for programming embedded systems?

- ☒ C
- ☐ Python
- ☐ Rust
- ☐ Assembly
- ☐ Java

The most popular language for programming embedded systems is **C** due to its efficiency, portability, and control over hardware. It allows developers to write code that directly interfaces with the microcontroller while maintaining a level of abstraction, which is crucial for working across different hardware platforms. While **Python** is great for scripting and high-level tasks, it's not typically used for low-level embedded systems programming due to performance limitations. **Java** is commonly used in application-level software development but is rarely used for embedded systems programming, except in platforms like Android. **Rust** is a newer language gaining traction for embedded systems, particularly due to its memory safety features, but it is not yet as widely adopted as C. **Assembly** language offers the most direct control over hardware but is more difficult to write, debug, and maintain, which limits its use in most embedded projects.

Correct answer



Question 15

0.25 / 0.25 pts

Can an embedded system perform more than one task or function?

- ☒ True
- ☐ False

Through proper use of timers and interrupts, you can configure your MCU to appear to execute many different tasks in parallel. This is how we build smart devices – by making a single MCU do many different, simple tasks in parallel.

Correct answer



Question 16

0.25 / 0.25 pts

What does the acronym NVIC stand for?

- ☒ Nested Vector Interrupt Controller
- ☐ Non-Volatile Interface Controller
- ☐ Nonlinear Voltage Integration Circuit
- ☐ Not Very Important Code

NVIC stands for **Nested Vector Interrupt Controller**, which is a key component in ARM Cortex-M microcontrollers. It manages interrupt handling, allowing the processor to respond efficiently to external and internal events. The NVIC supports nested interrupts, meaning higher-priority interrupts can preempt lower-priority ones, enabling real-time responsiveness. It is tightly integrated with the processor core to provide low-latency and deterministic interrupt handling, which is critical in embedded and real-time applications.

Correct answer



Question 17

0.25 / 0.25 pts

What are some ways that you can test your embedded code without a target (although limited)?

- ☒ Unit testing
- ☒ Emulators (in other words, emulating the MCU at the instruction level)
- ☒ Simulation (in other words, software model that mimics the MCU, including hardware devices)
- ☐ This is a dumb idea and shouldn't even be a question.

You should always strive to test your code automatically and with embedded code that relies on specific hardware, this requires unit testing small pieces of the code, simulating the code with a hardware simulator, and even running your code in an emulator. Even still, this cannot produce full test coverage for your application, but if you design this well, you can test many of the pieces of your application individually.

Correct answer



Question 18

0.25 / 0.25 pts

When designing a cyclic executive, how do you determine the optimal length of the major cycle based on the task periods?

- ☒ Major Cycle = Least Common Multiple (LCM) of all task periods
- ☐ Major Cycle = Average of all task periods
- ☐ Major Cycle = Sum of all task execution times
- ☐ Major Cycle = Maximum of all task periods
- ☐ Major Cycle = GCD of all task execution times

The **major cycle** in a cyclic executive represents the full schedule over which all periodic tasks are guaranteed to run at least once. To ensure that each task's execution pattern fits neatly within this cycle, the **Least Common Multiple (LCM)** of all task periods is used. This guarantees that all tasks align evenly within the schedule without drift, maintaining the correct timing for each task. Choosing the LCM ensures the schedule is predictable and repeatable—critical for timing accuracy in real-time embedded systems.



Question 19

0.2 / 0.25 pts

What are some examples of middleware in embedded applications?

- ☐ Real-Time Operating Systems (RTOS)
- ☒ File systems (for example, FATFS)
- ☒ Network stacks (for example, TCP/IP, MQTT libraries)
- ☒ USB stacks
- ☒ Bootloaders
- ☐ Interrupt vector tables
- ☐ Peripheral register definitions (for example, CMSIS headers)
- ☐ Compiler toolchains (for example, GCC, IAR)
- ☐ Debug probes (for example, J-Link, ST-Link)
- ☐ Power-on self-test routines

Middleware in embedded systems is the software layer that sits between the hardware and the application. It provides standardized services and capabilities to make development easier and more portable. Examples include the following:

- RTOS — Provides task scheduling and synchronization.
- File systems — Handle data storage and retrieval on flash or SD cards.

- Network stacks — Enable communication via protocols like TCP and IP, HTTP, or MQTT.
- USB stacks — Manage USB protocol layers and device classes.
- Bootloaders — Handle firmware updates and initial program loading.
- Incorrect options such as interrupt vector tables and CMSIS headers are either low-level or hardware-specific, and tools like compilers or debug probes are development tools, not middleware.

Correct answer



Question 20

0.25 / 0.25 pts

How do normal digital inputs and interrupt-based inputs differ?



Interrupt-based inputs automatically trigger a response when a signal change occurs, while normal digital inputs must be polled regularly.

- ☐ Normal digital inputs can only be read once per second.
- ☐ Interrupt-based inputs require manual polling to detect changes.
- ☐ Normal digital inputs are only used for analog signal detection.
- ☐ There is no difference—they function identically.

The main difference between **normal digital inputs** and **interrupt-based inputs** lies in how they detect and respond to signal changes. **Normal digital inputs** require **polling**, where the software periodically checks the input state, which can miss fast changes or consume CPU time. In contrast, **interrupt-based inputs** can automatically **trigger an interrupt service routine (ISR)** when a specific signal transition (rising edge, falling edge, or both) occurs. This allows for faster and more efficient response to events without constant monitoring, making them ideal for time-sensitive or asynchronous signals like button presses or external communication.

Correct answer



Question 21

0.25 / 0.25 pts

When designing a cyclic executive, how do you determine the optimal length of the minor cycle based on the task periods?

- ☒ Minor Cycle = Greatest Common Divisor (GCD) of all task periods
- ☐ Minor Cycle = Sum of all task periods
- ☐ Minor Cycle = (Maximum Task Execution Time) ÷ 2
- ☐ Minor Cycle = Least Common Multiple (LCM) of all task periods
- ☐ Minor Cycle = Average of all task execution times

When designing a **cyclic executive**, the **minor cycle** should be chosen as the **Greatest Common Divisor (GCD)** of all the task periods. This ensures that each task can be scheduled at a consistent interval that aligns with its required frequency. By using the GCD, you guarantee that every task can be broken down into an integer number of minor cycles, allowing a clean and predictable execution schedule. Choosing the GCD also minimizes the number of minor cycles in a major cycle, reducing scheduling complexity while maintaining precise timing for each task.

Correct answer



Question 22

0.25 / 0.25 pts

The total number of tasks in the **suspended** state can exceed the total number of processors available in the system.

- ☒ True
- ☐ False

A system can only run as many tasks as it has processors, but many more tasks can be loaded into the scheduler.

Correct answer



Question 23

0.25 / 0.25 pts

What is preemption?

- ☒ The scheduler forcibly interrupts a running task to switch to a higher-priority task.
- ☐ The act of delaying task creation until system idle time.
- ☐ A mechanism where a task voluntarily gives up the CPU.
- ☐ A process where the CPU executes tasks in the order they are created.
- ☐ A feature that disables all interrupts during task execution.

Preemption is when the scheduler **forcibly interrupts a currently running task** in order to start running a **higher-priority task** that just became ready. This ensures that the most critical tasks are given CPU time as soon as they need it.



Question 24

0.08 / 0.25 pts

What are three possible approaches to increasing the level of safety for a **flat** operating system?

- ☒ Implement kernel and process privilege separation.
- ☒ Integrate memory protection into the scheduler.
- ☐ Provide a system call API through the kernel to access resources.
- ☒ Use cooperative multitasking instead of preemption.

- ☐ Add more task priorities beyond fixed levels.
- ☐ Allow tasks unrestricted access to all resources.
- ☐ Rely solely on hardware watchdog timers for safety.
- ☐ Increase the number of interrupts in the system.

Increasing safety in a **flat** operating system involves **implementing kernel and process privilege separation** to restrict what tasks can do, **integrating memory protection into the scheduler** to enforce isolation, and **providing a system call API through the kernel** to safely control access to system resources. Other options either don't improve safety or could weaken system reliability.

Correct answer



Question 25

0.25 / 0.25 pts

When a task is created, the TCB is created and the task is immediately executed.

- ☐ True
- ☒ False

When a task is created, the **Task Control Block (TCB)** is indeed allocated and initialized to store the task's context, but the task is **not immediately executed**. Instead, it is placed into the **ready state**, and whether it runs immediately depends on the **scheduler's decision**, which is based on the task's **priority** and the state of other tasks. If a higher-priority task is already running, the new task will wait. So while task creation sets everything up, **execution only begins when the scheduler allows it**.

Correct answer



Question 26

0.25 / 0.25 pts

What is provided by the CMSIS-Core system_<device>.c source file?

- ☒ System initialization code, including SystemInit() and SystemCoreClockUpdate(), to configure the clock system and core settings
- ☐ The interrupt vector table and reset handler that sets up the runtime environment
- ☐ A set of peripheral driver functions for UART, SPI, and I2C communication
- ☐ A real-time operating system scheduler and thread management functions
- ☐ A library of digital signal processing functions optimized for Cortex-M processors

The system_<device>.c file is part of CMSIS-Core and contains system-level initialization code specific to the microcontroller. It typically defines the SystemInit() function, which configures the system clock and other low-level settings, and SystemCoreClockUpdate(), which updates the global SystemCoreClock variable to reflect the current core clock frequency. This file ensures the processor is correctly configured before the application code runs.

Correct answer



Question 27

0.25 / 0.25 pts

ChibiOS is primarily optimized for:

- ☒ Small footprint and context switching performance
- ☐ Efficient for resource-constrained embedded systems
- ☐ Modular and portable across various microcontrollers
- ☐ Supports preemptive multitasking with fast context switching
- ☐ Includes integrated device drivers and middleware

ChibiOS is primarily optimized for embedded applications requiring a small memory footprint combined with fast context switching capabilities. It is designed to run efficiently on resource-limited microcontrollers, offering modularity and portability across many platforms. Its kernel supports preemptive multitasking with fast context switches, making it suitable for time-critical embedded systems. Additionally, ChibiOS provides built-in device drivers and middleware, simplifying development in constrained environments.

Correct answer



Question 28

0.25 / 0.25 pts

Riot OS implements its own TCP/IP stack.

- ☒ True
- ☐ False

Riot OS includes its own lightweight and modular TCP/IP stack called GNRC. This stack is designed specifically for constrained devices, offering IPv6 support, multi-threaded processing, and extensibility. It allows Riot OS to operate efficiently in low-power, low-memory environments while still supporting standard Internet communication protocols.

Correct answer



Question 29

0.25 / 0.25 pts

FreeRTOS offers 5 different flavors of heap memory management. Heap 3 is optimized for:

- ☒ Using the standard C library malloc() and free() functions
- ☐ Relies on the platform's native heap management
- ☐ Supports full dynamic allocation and deallocation
- ☐ No internal memory management by FreeRTOS
- ☐ Best when platform-provided allocators are preferred

Heap 3 in FreeRTOS simply wraps the standard C library's `malloc()` and `free()` functions. It is optimized for systems where the developer prefers to rely on the platform's built-in dynamic memory management rather than FreeRTOS's own heap implementations. This makes it easy to integrate but depends heavily on the quality and predictability of the underlying C library's allocator.

Correct answer



Question 30

0.25 / 0.25 pts

In SPI communications, what does clock phase indicate?

- ☒ Whether data is sampled on the leading or trailing edge of the clock signal
- ☐ Whether the clock idles high or low
- ☐ The number of bits transmitted per clock cycle
- ☐ Whether the master or slave initiates communication
- ☐ The voltage level of the data lines

Clock phase (CPHA) in SPI determines when data is sampled and shifted relative to the clock edges. Specifically, it defines whether data is captured on the first (leading) or second (trailing) clock edge after the chip select line is activated. This setting must match between the master and slave to ensure correct data interpretation, especially when combined with clock polarity (CPOL), which defines the idle state of the clock.

Correct answer



Question 31

0.25 / 0.25 pts

If you want to add devices to an I2C bus, what do you need to ensure?

- ☒ Each device has a unique address on the bus.
- ☐ Each device has a dedicated data line.
- ☐ The master provides a separate clock line for each device.
- ☐ All devices are connected using differential signaling.
- ☐ Each device has its own pull-up resistor on the SDA line.

In I2C communication, all devices share the same SDA (data) and SCL (clock) lines. To communicate reliably, each device must have a unique 7-bit or 10-bit address so the master can distinguish between them. Address conflicts can prevent proper communication or cause data corruption. Some devices allow configurable addresses via hardware pins or software settings, while others have fixed addresses, which can limit how many of the same type can be used on a single bus.

Correct answer



Question 32

0.25 / 0.25 pts

In SPI communications, what is a method in which the master can keep the clock signal alive while waiting for the slave to finish sending data?

- ☒ The master continues sending dummy bytes to generate clock pulses
- ☐ The master lowers the chip select line and waits
- ☐ The slave requests more clock cycles using a control line
- ☐ The master switches to half-duplex mode
- ☐ The slave generates its own clock pulses

In SPI, the master is responsible for generating the clock signal. If the master needs to wait for the slave to finish preparing data (e.g., in a read operation), it can send dummy bytes (often 0xFF or 0x00) to keep the clock active. This allows the slave to shift out its data on the MISO line in sync with the clock pulses. This technique is commonly used in SPI flash memory and sensor communications.

Wrong answer



Question 33

0 / 0.25 pts

How does I2C enable multi-master operation?

- ☐ By using arbitration on the SDA line and clock synchronization on the SCL line
- ☐ By assigning unique addresses to each master to avoid collisions
- ☐ By using separate data and clock lines for each master
- ☒ By allowing only one master to be active at a time through software control
- ☐ By implementing a round-robin token-passing protocol among masters

I2C enables multi-master operation through two key mechanisms: arbitration and clock synchronization. During transmission, each master monitors the SDA line while sending data. If a master tries to send a high (logic 1) but detects a low (logic 0), it knows another master is asserting control and it immediately stops transmitting, thus losing arbitration. This ensures that only one master continues without corrupting the bus. Simultaneously, all masters contribute to the SCL line using open-drain outputs, and the effective clock is the logical AND of all masters' clocks. This clock synchronization ensures that all devices operate in unison, even if their internal clocks differ slightly.

Wrong answer



Question 34

0 / 0.25 pts

What is the maximum quantization error for a 12-bit ADC with a 3.3 V reference voltage?

- ☐ 0.4 mV
- ☒ 0.8 mV

- ☐ 1.6 mV
- ☐ 3.3 mV
- ☐ 6.6 mV

First, calculate the resolution: $\text{Resolution} = 3.3 / 2^{12} = 3.3 / 4096 \approx 0.805 \text{ mV}$. The maximum quantization error is half the resolution: $0.805 / 2 \approx 0.4 \text{ mV}$. This is the largest possible deviation between the actual analog input and its digital representation.

Correct answer



Question 35

0.25 / 0.25 pts

What is quantization error in the context of ADCs?

- ☒ The difference between the actual analog value and the nearest digital representation
- ☐ The time delay between sampling and conversion
- ☐ The noise introduced by the analog signal
- ☐ The error caused by incorrect reference voltage
- ☐ The mismatch between input and output impedance

Quantization error arises because an ADC maps a continuous range of analog values to a finite number of digital levels. This means each digital value represents a small range of analog inputs. The difference between the actual analog input and the center of the digital step it falls into is the quantization error. The worst-case error is half the resolution step.

Correct answer



Question 36

0.25 / 0.25 pts

What is the effect of increasing the sampling rate without increasing bit depth?

- ☒ Improves time resolution but not amplitude resolution
- ☐ Improves both time and amplitude resolution
- ☐ Reduces quantization noise
- ☐ Increases quantization error
- ☐ Has no effect on signal quality

Increasing the sampling rate means the analog signal is measured more frequently, which improves the time resolution—how accurately the signal's changes over time are captured. However, if the bit depth (number of bits per sample) remains the same, the amplitude resolution does not improve. The signal is still quantized into the same number of levels, so the precision of each sample remains unchanged.

Correct answer



Question 37

0.25 / 0.25 pts

What is the primary function of an Analog-to-Digital Converter (ADC) in a microcontroller system?

- ☒ Converts analog signals into digital values for processing
- ☐ Converts digital signals into analog outputs
- ☐ Amplifies analog signals for better resolution
- ☐ Filters noise from analog inputs
- ☐ Stores analog signals in memory

An ADC is essential in embedded systems because most real-world signals—like temperature, sound, or light—are analog. Microcontrollers, however, operate digitally. The ADC bridges this gap by converting continuous analog voltages into discrete digital values that the microcontroller can read and process. This enables digital systems to interact with the analog world.

Correct answer



Question 38

0.5 / 0.5 pts

Why are coprocessors more power-efficient than general-purpose cores for specific tasks?

- ☒ They are optimized for specific functions and operate at lower frequencies.

Coprocessors are designed for specific operations (e.g., FFTs, encoding) and can perform them more efficiently than a general-purpose CPU, both in terms of power and performance.

- ☐ They use more memory.
- ☐ They require fewer instructions.
- ☐ They are easier to program.
- ☐ They eliminate the need for DMA.

Correct answer



Question 39

0.5 / 0.5 pts

What is the primary purpose of synchronization in multicore systems?

- ☒ To coordinate access to shared resources and ensure correct program execution.

Synchronization is essential in multicore systems to manage how multiple threads or processes interact with shared resources such as memory or I/O devices. Without proper synchronization, concurrent access can lead to race conditions, data corruption, and unpredictable behavior. Synchronization mechanisms like locks, barriers, and atomic operations ensure that only one thread accesses critical sections at a time, preserving data integrity and program correctness.

- ☐ To increase the clock speed of each core.

- ☐ To reduce the number of processor cores needed.
- ☐ To eliminate the need for memory management.
- ☐ To allow simultaneous execution of identical instructions.

Correct answer



Question 40

0.5 / 0.5 pts

In burst mode DMA operation, what happens during a data transfer?

- ☒ The DMA controller retains control of the bus until the entire data block is transferred.

In burst mode, the DMA controller takes full control of the system bus and completes the entire data transfer in one uninterrupted sequence before releasing the bus back to the CPU. This mode is highly efficient for transferring large blocks of data because it minimizes the overhead of repeatedly requesting and releasing the bus. However, it can temporarily block the CPU from accessing memory, which may not be ideal in systems with tight real-time constraints. This trade-off between throughput and bus availability is a key consideration when choosing between burst and single-cycle DMA modes.

- ☐ The CPU and DMA controller alternate control of the bus for each byte.
- ☐ The DMA controller transfers only one byte before releasing the bus.
- ☐ The CPU must acknowledge each byte transferred.
- ☐ The DMA controller requires manual intervention after each transfer.

Correct answer



Question 41

0.5 / 0.5 pts

What is the main advantage of using load-reserved/store-conditional over test-and-set?

- ☒ It reduces contention and improves scalability in multicore systems.

Load-reserved/store-conditional operations are designed to be more scalable than test-and-set, especially in systems with many cores. They avoid locking the memory bus and instead rely on cache coherence mechanisms to detect conflicts. This reduces contention and improves performance in high-concurrency environments.

- ☐ It guarantees stronger memory consistency.
- ☐ It eliminates the need for memory barriers.
- ☐ It simplifies compiler design.
- ☐ It avoids the need for hardware support.

Correct answer



Question 42

0.5 / 0.5 pts

Which of the following is a lock-free synchronization technique?

- ☒ Hardware queues

Hardware queues enable lock-free communication between cores or between cores and devices. By decoupling producers and consumers, they eliminate the need for mutual exclusion, reducing overhead and improving throughput in systems that require high-speed data exchange.

- ☐ Spinlocks
- ☐ Mutexes
- ☐ Semaphores
- ☐ Memory fences

Correct answer



Question 43

0.5 / 0.5 pts

What is the main benefit of using hardware queues in multicore systems?

- ☒ They allow lock-free communication between cores and devices.

Hardware queues provide a mechanism for decoupling producers and consumers of data, allowing them to operate independently without requiring locks. This is especially useful in multicore systems where multiple cores or devices need to exchange data efficiently. By avoiding locks, hardware queues reduce contention and improve system throughput.

- ☐ They increase the size of the instruction cache.
- ☐ They reduce the need for memory fences.
- ☐ They eliminate the need for DMA.
- ☐ They simplify compiler design.

Wrong answer



Question 44

0 / 0.5 pts

What is the main drawback of using a Gaussian or Blackman filter compared to a moving average filter?

- ☐ They require convolution and are computationally slower
- ☐ They introduce nonlinear phase distortion
- ☐ They cannot be used for noise reduction
- ☐ They are not suitable for image processing
- ☒ They require recursive implementation

Correct answer



Question 45

0.5 / 0.5 pts

What is the main reason overshoot in a step response is undesirable?

- ☒ It distorts the amplitude of the signal

Overshoot causes the output signal to exceed the actual value of the step, introducing distortion. This can misrepresent the true amplitude of the signal and lead to errors in interpretation or further processing. In time-domain applications, preserving the integrity of amplitude transitions is critical. Therefore, filters are often designed to minimize or eliminate overshoot (Chapter 14, p. 267).

- ☐ It improves frequency resolution
- ☐ It increases the filter's gain
- ☐ It enhances the filter's phase linearity
- ☐ It reduces the filter's execution time

Correct answer



Question 46

0.5 / 0.5 pts

What is the cumulative distribution function (cdf) of a Gaussian distribution?

- ☒ The probability that a value is less than or equal to a given point

The cdf of a Gaussian distribution gives the probability that a random variable is less than or equal to a specific value. It is obtained by integrating the Gaussian pdf from negative infinity to that point. The cdf is useful for calculating probabilities over intervals and is often tabulated for standard normal distributions. This is shown in Figure 2-9 and Table 2-5 (Chapter 2, p. 29).

- ☐ The derivative of the Gaussian pdf
- ☐ The histogram of a Gaussian signal
- ☐ The area under the histogram
- ☐ The frequency response of a Gaussian filter

Correct answer



Question 47

0.5 / 0.5 pts

What is the main advantage of using decibels (dB) in filter analysis?

- ☒ They allow easy comparison of large and small values

Decibels provide a logarithmic scale that makes it easier to compare values that span several orders of magnitude. This is especially useful in frequency response plots, where both small and large gains or attenuations need to be visualized. Decibels also simplify the math when combining gains or losses in cascaded systems. This is explained in Chapter 14, p. 264.

- ☐ They eliminate the need for frequency response plots
- ☐ They simplify time-domain analysis
- ☐ They convert amplitude to phase

- ☐ They remove the effects of noise

Correct answer



Question 48

0.5 / 0.5 pts

What does stopband attenuation measure?

- ☒ How much unwanted frequencies are suppressed

Stopband attenuation quantifies how effectively a filter suppresses frequencies outside the desired range. Higher attenuation means less leakage of unwanted signals into the output. This is critical in applications like communication systems, where interference must be minimized. It is illustrated in Figure 14-4 (Chapter 14, p. 269).

- ☐ How fast the filter responds to a step input
- ☐ How much the filter delays the signal
- ☐ How wide the passband is
- ☐ How linear the phase response is

Correct answer



Question 49

0.5 / 0.5 pts

What is the main disadvantage of using Gaussian or Blackman filters compared to the moving average filter?

- ☒ They require convolution, which is computationally expensive

Gaussian and Blackman filters offer better stopband attenuation but must be implemented using convolution, which is significantly slower than the recursive implementation of the moving average filter (Chapter 15, p. 282).

- ☐ They introduce nonlinear phase distortion
- ☐ They cannot be used for smoothing
- ☐ They have poor stopband attenuation
- ☐ They are not suitable for time-domain signals

Correct answer



Question 50

0.5 / 0.5 pts

Which of the following best describes the behavior of `xSemaphoreTake()` with a timeout?

- ☒ It blocks the calling task until the semaphore is available or the timeout expires.

`xSemaphoreTake()` is used to acquire a semaphore. If the semaphore is not available, the calling task can optionally block for a specified number of ticks. If the semaphore becomes available during that time, the task proceeds; otherwise, it times out and returns an error. This is essential for synchronizing access to shared resources. (Reference: Ivan Cibrario Bertolotti, p. 123)

- ☐ It immediately returns an error if the semaphore is unavailable.
- ☐ It deletes the semaphore after use.
- ☐ It suspends the task indefinitely.
- ☐ It increases the semaphore count.

Correct answer



Question 51

0.5 / 0.5 pts

Which of the following is a valid reason to use a queue set in FreeRTOS?

- ☒ To allow a task to block on multiple queues or semaphores simultaneously.

Queue sets allow a task to wait on multiple queues or semaphores at once. This is useful when a task needs to respond to different types of events or data sources but wants to avoid polling each queue individually. The task blocks on the set and is unblocked when any member queue or semaphore becomes active. (Reference: Mastering FreeRTOS, p. 121)

- ☐ To increase the maximum size of a queue.
- ☐ To enable recursive locking of semaphores.
- ☐ To allow ISRs to access multiple queues.
- ☐ To implement priority inheritance.

Correct answer



Question 52

0.5 / 0.5 pts

What is the effect of calling `xQueueOverwrite()` on a queue with a length of 1?

- ☒ It replaces the existing item in the queue, even if the queue is full.

`xQueueOverwrite()` is designed for queues of length 1 and is used to ensure that the most recent data is always available. If the queue already contains an item, that item is discarded and replaced with the new one. This is useful in scenarios like sensor data updates, where only the latest value matters. Reference: Mastering FreeRTOS, p. 132

- ☐ It blocks the task until the queue is empty.
- ☐ It appends the item to the back of the queue.
- ☐ It causes a queue reset.
- ☐ It returns an error if the queue is full.

Correct answer



Question 53

0.5 / 0.5 pts

What is the purpose of the `vApplicationMallocFailedHook()` function?

- ☒ To handle memory allocation failures by executing user-defined recovery code.

If dynamic memory allocation fails (e.g., due to insufficient heap), FreeRTOS calls `vApplicationMallocFailedHook()` if it is defined. This allows the developer to log the error, reset the system, or take other corrective actions. (Reference: Mastering FreeRTOS, p. 3.3.6)

- ☐ To allocate memory for tasks.
- ☐ To free unused memory.
- ☐ To reset the heap.
- ☐ To suspend the idle task.

Correct answer



Question 54

0.5 / 0.5 pts

What is the purpose of the `xTimerStart()` function?

- ☒ To start a software timer so that it begins counting toward its expiration.

`xTimerStart()` activates a software timer that was previously created. Once started, the timer counts down based on its configured period and triggers its callback function when it expires. (Reference: Mastering FreeRTOS, p. 6.5.2)

- ☐ To create a new timer.
- ☐ To reset the system tick.
- ☐ To delay a task.
- ☐ To suspend the scheduler.

Correct answer



Question 55

0.5 / 0.5 pts

What is the role of the Idle Hook function in FreeRTOS?

- ☒ It allows the user to execute custom code when the idle task runs.

The idle hook is an optional user-defined function that is called by the idle task each time it runs. It can be used for background processing, low-priority tasks, or entering low-power modes. To use it, `configUSE_IDLE_HOOK` must be set to 1 in `FreeRTOSConfig.h`. (Reference: Mastering FreeRTOS, p. 75)

- ☐ It handles memory allocation failures.
- ☐ It is used to implement mutual exclusion.
- ☐ It is required for all FreeRTOS applications.
- ☐ It replaces the scheduler.

Correct answer



Question 56

0.5 / 0.5 pts

Which of the following is a benefit of using `xEventGroupSync()` over `xEventGroupWaitBits()`?

- ☒ It allows a task to both set and wait on bits in a single atomic operation.

`xEventGroupSync()` is designed for task synchronization at a barrier. It allows a task to set its own bit and wait for all required bits to be set in a single call, making it ideal for phase-based coordination. (See Chapter 9, Section 9.4.1, page 87)

- ☐ It supports queuing of multiple values.
- ☐ It can be used from ISRs.
- ☐ It clears all bits automatically.
- ☐ It is faster than task notifications.

Correct answer



Question 57

0.5 / 0.5 pts

What is the purpose of the `ulBitsToWaitFor` parameter in `xEventGroupWaitBits()`?

- ☒ It specifies which bits the task is waiting to become set.

The `ulBitsToWaitFor` parameter defines the bit mask of the event bits the task is interested in. The task will block until the specified bits are set, depending on whether it is waiting for any or all of them. (See Chapter 9, Section 9.3.4, page 84)

- ☐ It defines the priority of the waiting task.
- ☐ It sets the bits that will be cleared on exit.
- ☐ It determines the timeout duration.
- ☐ It resets the event group before waiting.

Wrong answer



Question 58

0 / 0.5 pts

Which of the following best describes the behavior of `ulTaskNotifyTake()`?

- ☐ It blocks a task until its notification value is non-zero, then decrements it.
- ☒ It sends a notification to another task.
- ☐ It waits for a specific event group bit.
- ☐ It creates a task notification object.
- ☐ It clears all task notifications.

Correct answer



Question 59

0.5 / 0.5 pts

Which of the following is a reason to prefer task notifications over semaphores?

- ☒ They use less RAM and are faster.

Task notifications are built into each task and do not require separate kernel objects, making them more memory-efficient and faster than semaphores. However, they are limited to one per task and are not suitable for all use cases. (See Chapter 10, Section 10.2.1, page 91)

- ☐ They support multiple readers.
- ☐ They allow queuing of data.
- ☐ They can be used from multiple tasks simultaneously.
- ☐ They automatically reset the task's priority.

Correct answer



Question 60

0.5 / 0.5 pts

What is the effect of setting `xClearOnExit` to `pdTRUE` in `xEventGroupWaitBits()`?

- ☒ The bits being waited on will be cleared automatically when the function returns.

The `xClearOnExit` parameter in `xEventGroupWaitBits()` determines whether the bits that caused the function to return should be cleared automatically. Setting it to `pdTRUE` ensures that once the task is unblocked, the relevant bits are cleared, preventing other tasks from acting on stale events. (See Chapter 9, Section 9.3.4, page 84)

- ☐ The event group will be deleted after the wait.
- ☐ The task will be suspended after the wait.
- ☐ The event group will be reset to zero.
- ☐ The task will yield to the idle task.

Correct answer



Question 61

0.5 / 0.5 pts

Which of the following best describes the use of `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)`?

- ☒ It blocks the task indefinitely until a notification is received, then clears the notification value.

Calling `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)` causes the task to block indefinitely until its notification value becomes non-zero. The `pdTRUE` argument ensures the value is cleared to zero after the function returns, mimicking the behavior of a binary semaphore. (See Chapter 10, Section 10.3.4, page 94)

- ☐ It sends a notification to another task and clears its own.
- ☐ It waits for all event group bits to be cleared.
- ☐ It resets the task's priority.
- ☐ It creates a new task notification object.

Quiz Score: 24.28 out of 40

* Some questions not yet graded